

LIST OF CONTENTS

CHAPTER NO	TITLE	PG. NO
	List of Figures	3
	Abstract	4
1	INTRODUCTION	5
2	LITERATURE SURVEY	7
	2.1 Introduction	7
	2.2 Existing System	8
3	SYSTEM ANALYSIS	10
	3.1 Existing System	10
	3.1.1 Disadvantages	10
	3.2 Proposed System	10
	3.2.1 Advantages	11
4	FEASIBILITY STUDY	12
	4.1 Technical Feasibility	12
	4.2 Economic Feasibility	12
	4.3 Operational Feasibility	12
	4.4 Legal and Ethical Feasibility	14
5	SYSTEM DESIGN	16
	5.1 MTCNN	16
	5.2 UML Diagram	17
	5.3 Class Diagram	20
	5.4 Data Flow Diagram	21
	5.5 Usecase Diagram	22
	5.6 Sequence Diagram	23
	5.7 Flow Chart	24
6	SYSTEM IMPLEMENTATION	25
	6.1 Technology Description	25
	6.2 Modular Description	39
7	TESTING AND DEBUGGING	42
	7.1 Testing Methodologies	42
	7.2 Test Cases	47

	7.3 Output Screens	48
8	CONCLUSION	53
9	FUTURE ENHANCEMENT	54
10	BIBILIOGRAPHY	55

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Fig 5.1	System Architecture	16
Fig 5.3	Class Diagram	20
Fig 5.4	Data Flow Diagram	21
Fig 5.5	Use case Diagram	22
Fig 5.6	Sequence Diagram	23
Fig 5.7	Flow Chart	24
Fig 6.1	java programming	26
Fig 6.2	java program	26
Fig 6.3	java plat form	27
Fig 6.4	java ide	29
Fig 6.5	java compiler	34
Fig 6.6	networking TCP/IP	35
Fig 6.7	total address	36
Fig 6.8	Tomcat Webserver	39
Fig 7.1	Levels of Testing	43
Fig 7.2	Test case Template	47

ABSTRACT

In elastic optical networks, digital coherent transceivers modify their symbol rate, modulation format, and forward error correction to best serve the network demands. In a nonlinear elastic optical network, these parameters are inherently coupled with the routing algorithm. We propose to use congestion aware routing in a nonlinear elastic optical network and demonstrate its efficacy for the NSFNET reference network (14 nodes, 22 links). The network is sequentially loaded with 100 GbE demands until a demand becomes blocked, this procedure being repeated 10 000 times to estimate the network blocking probability (NBP). Three routing algorithms are considered: 1) shortest path routing; 2) simple congestion aware algorithm; and 3) weighted congestion aware routing algorithm with 50, 25, 12.5, and 6.25 GHz resolution flexgrids. For $\text{NBP} = 1\%$ using a 50 GHz grid, congestion aware routing doubles the network capacity compared with the shortest path routing. When congestion aware routing is combined with a 6.25 GHz resolution flexgrid, a fivefold increase in network capacity is afforded.

CHAPTER - 1

INTRODUCTION

Congestion aware routing in nonlinear elastic optical networks is an important topic in modern optical communication systems, especially with the rapid growth of internet traffic and high-bandwidth applications. Optical networks form the backbone of today's communication infrastructure, supporting services such as cloud computing, video streaming, and data centers. Traditional optical networks use fixed grid technology, where spectrum allocation is rigid and often leads to inefficient utilization of available bandwidth. To overcome these limitations, elastic optical networks (EONs) have been introduced, which allow flexible allocation of spectrum resources based on the actual demand of data traffic.

In elastic optical networks, bandwidth can be dynamically assigned using flexible spectrum slots, improving efficiency and accommodating different types of services. However, as traffic demand continues to grow, congestion becomes a major challenge. Congestion occurs when multiple data flows compete for limited network resources, leading to increased blocking probability, delays, and reduced quality of service. Additionally, optical networks are affected by nonlinear physical impairments such as signal distortion, noise, and interference, which further degrade performance, especially over long distances or under high load conditions.

Congestion aware routing aims to address these issues by selecting paths that not only consider shortest distance or minimum hops but also take into account network congestion levels and physical layer impairments. By analyzing current traffic conditions and resource availability, the routing algorithm can avoid heavily congested links and distribute traffic more evenly across the network. This helps in reducing blocking probability, improving network utilization, and ensuring better quality of transmission.

Another important aspect is the consideration of nonlinear effects in optical fibers, such as self-phase modulation and cross-phase modulation, which can impact signal quality. Advanced routing strategies incorporate these factors into their decision-making process to ensure reliable data transmission. Techniques such as adaptive modulation, spectrum assignment, and load balancing are often used in combination with congestion aware routing to optimize overall network performance.

Overall, congestion aware routing in nonlinear elastic optical networks plays a vital role in enhancing the efficiency, reliability, and scalability of modern communication systems.

It enables better utilization of spectrum resources, reduces network congestion, and improves the quality of service for end users. This approach is essential for supporting future high-speed communication demands and building intelligent, flexible, and resilient optical networks.

CHAPTER - 2

LITERATURE SURVEY

2.1 INTRODUCTION: -

Congestion aware routing in nonlinear elastic optical networks focuses on improving how data is transmitted in modern optical communication systems. Elastic optical networks allow flexible use of bandwidth by allocating spectrum based on demand, which makes them more efficient than traditional fixed networks. However, as network traffic increases, congestion becomes a major issue, causing delays, data loss, and reduced performance. At the same time, nonlinear effects in optical fibers, such as signal distortion and interference, can further affect the quality of transmission. To overcome these problems, congestion aware routing selects paths by considering both network traffic conditions and signal quality. It avoids heavily congested routes and distributes data more efficiently across the network. This helps in reducing delays, improving resource utilization, and maintaining better communication quality. Overall, this approach makes optical networks more reliable and efficient for handling high-speed data communication.

Key Objectives:

- To design routing strategies that avoid heavily loaded paths and distribute traffic efficiently across the network.
- To consider nonlinear effects and signal impairments while selecting routes to ensure better data transmission quality.
- To efficiently allocate flexible spectrum resources in elastic optical networks based on traffic demand.
- To reduce blocking probability, delays, and data loss, ensuring stable and high-speed communication.

Importance of the Project:

This project is important because it improves the efficiency and reliability of modern optical communication networks by addressing both congestion and nonlinear signal impairments. As data traffic continues to grow rapidly, managing network congestion becomes essential to maintain high performance. By using congestion aware routing, the

system can distribute traffic more effectively, reduce delays, and improve overall quality of service. It also ensures better utilization of available spectrum resources in elastic optical networks. This approach is crucial for supporting high-speed applications such as cloud computing, video streaming, and data centers, making communication networks more scalable and future-ready.

2.2 EXISTING SYSTEM: -

- As OPTICAL networks move towards being able to cope with elastic demands the traditional challenge of routing and wavelength assignment becomes replaced by that of routing, modulation and spectrum allocation (RMSA).
- The high dimensionality of the RMSA problem can however be simplified due to the coupling between the dimensions caused by the fiber nonlinearities.

Challenges in Existing Systems:

- Algorithms like Ullmann's do not use indexing structures, making them inefficient and time-consuming for large-scale graph databases.
- Most existing methods focus on exact subgraph matching and do not consider set similarity, reducing flexibility in real-world scenarios.
- Since set similarity is not integrated into the matching process, additional computation is required after matching, increasing processing time.
- Techniques like graph alignment and join-based methods may struggle with performance and scalability when handling very large graphs.

Technologies Used in Existing Systems:

- **High Complexity of RMSA Problem**
Routing, modulation, and spectrum allocation (RMSA) together create a high-dimensional and complex optimization problem.
- **Impact of Fiber Nonlinearities**
Nonlinear effects in optical fibers complicate decision-making and affect signal quality, making routing more challenging.

- **Difficulty in Optimal Resource Allocation**

Efficiently allocating spectrum and selecting modulation formats under varying network conditions is difficult.

- **Scalability Issues in Large Networks**

As network size and traffic increase, handling RMSA efficiently becomes more computationally intensive.

CHAPTER - 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM: -

- As OPTICAL networks move towards being able to cope with elastic demands the traditional challenge of routing and wavelength assignment becomes replaced by that of routing, modulation and spectrum allocation (RMSA).
- The high dimensionality of the RMSA problem can however be simplified due to the coupling between the dimensions caused by the fiber nonlinearities.

3.1.1 DISADVANTAGES:

In a nonlinear optical network, the RMSA becomes primarily a routing problem, with the assignment of modulation format and forward error correction (FEC) depending on the signal to noise available for a given route and the optical spectrum to be assigned depending on the required data rate.

Technologies Used in Existing Systems:

- **High Complexity of RMSA Problem** - Routing, modulation, and spectrum allocation (RMSA) together creates a high-dimensional and complex optimization problem.
- **Impact of Fiber Nonlinearities** - Nonlinear effects in optical fibres complicate decision-making and affect signal quality, making routing more challenging.
- **Difficulty in Optimal Resource Allocation** - Efficiently allocating spectrum and selecting modulation formats under varying network conditions is difficult.
- **Scalability Issues in Large Networks** - As network size and traffic increase, handling RMSA efficiently becomes more computationally intensive.

3.2 PROPOSED SYSTEM: -

- For a pair of nodes, select the shortest path that avoids the link with the highest spectral usage (determined by measuring the total optical power which is proportional to spectral usage).
- For this path determine the total number of amplifier spans (100 km herein) in order to determine the received signal to noise ratio (SNR).

- For this SNR, determine the maximum net spectral efficiency (NSE) based on known relationship between SNR and NSE for a range of polarization division multiplexed formats with Nyquist spectra where variable rate FEC is also included.
- Finally determine the gross symbol rate and assign spectrum to serve the demand between the two nodes.

3.2.1 ADVANTAGES

We focus on the number of demands that cause blocking within the network in 1% of occasions. Given that the network blocking occurs when blocking occurs in the most congested link, in essence we are concerned with the distribution of a minimum.

Key Features of the Proposed System:

- **Congestion-Aware Path Selection**

The system selects routes by avoiding links with high spectral usage, reducing congestion and improving network performance.

- **SNR-Based Transmission Optimization**

Calculates the signal-to-noise ratio (SNR) based on the number of amplifier spans to ensure reliable communication.

- **Adaptive Spectral Efficiency Selection**

Determines the maximum net spectral efficiency (NSE) using the relationship between SNR and modulation formats.

- **Dynamic Spectrum Allocation**

Allocates spectrum based on demand and calculated symbol rate, ensuring efficient utilization of network resources.

- **Integration of Advanced Modulation and FEC Techniques**

Uses polarization division multiplexing and variable rate forward error correction (FEC) to enhance data transmission quality and capacity.

CHAPTER - 4

FEASABILITY STUDY

To manner studies and explores of an active and scientific nature and to encourage the conversation and enlargement of approaches and tools for operational analysis is useful to protection problems.

Feasibility study is accompanied once the difficult is obviously understood. The feasibility study which is a great level lozenge version of the whole system analysis and design procedure. The independent is to define whether the planned system is possible or not and it benefits us to the least expense of how to resolve the problem and to govern, if the Problem is wealth solving.

The following are the three important tests that have been conceded out for feasibility study.

- Technical Feasibility
- Economic feasibility
- Operational feasibility

4.1 Technical feasibility

In the technical feasibility study, one has to assess whether the implemented system can be established using existing technology or not. It is intended to implement the implemented system in JSP. The project enabled is theoretically feasible since the following reasons.

All needed technology exists to improve the system.

The existing system is so malleable that it can be advanced further.

4.2 Economic feasibility

As a portion of this, the expenses and profits related with the implemented systems are to be associated. The project is carefully feasible only if tangible and intangible assistances balance the cost. We can say the implemented system is feasible founded on the following grounds.

The charge of developing the filled system is sensible.

The cost of hardware and software for the application is less.

4.3 Operational feasibility

This project is operationally feasible for there is necessary support from the project organization and the users of the implemented system. Implemented system absolutely

does not damage and determination not create the corrupt results and no problem will ascend after implementation of the system.

Cost estimation and scheduling

The cost and schedule estimation process helps in determining number of resources to complete all project activities. It generally involves approximation and development of costing alternatives to plan, perform or work, deliver, or give project. A good estimation is very much essential for keeping a project under budget.

Two perspectives are generally required to derive project plans. These perspectives are given below:

1. Forward-Looking:

The Forward-Looking approach is also known as Top-Down approach. This approach generally starts with describing and explaining various project tasks that involve starting with project aim or end deliverable and breaking it all down into smaller planning chunks. Top-down budgeting also refers to a method of budgeting where project managers prepare a high-level budget for organization.

These project managers or senior management develops and creates a characterization of overall size, process, environment, people, and quality that is essential for software project. In this approach, duration of deliverable's is estimated.

It generally takes less time and effort than bottom-up estimate. With help of software cost estimation model, an estimation of overall effort and schedule is done. The project manager generally divides estimation of overall effort into a top-level of WBS (Work Breakdown Structure).

They also divide schedule into major milestones dates. At this stage, sub-project managers are simply given responsibility for decomposing every element of WBS into lower levels with help of various allocations of top-level, staffing profile, and, major milestones dates as constraints.

The main benefit of this approach is use of holistic data from earlier projects or products, along with unmitigated risks, and scope creeps. This also helps in reducing risk of overlooked work activities or costs.

2. Backward-Looking:

Backward-Looking approach is also known as Bottom-up approach.

In this approach, project team breaks requirements of clients down, determining lowest level appropriate to develop a range of estimates, covering overall scope of project based on available definition of task.

Overall elements of lowest level WBS are generally explained into detailed tasks, for which WBS element manager is responsible for estimating budget and schedule.

All of these estimates are joined and integrated into higher-level WBS budgets and milestones.

4.4 Legal & Ethical Feasibility

1. Compliance with Legal Regulations

A fraud detection system must adhere to financial and data protection laws, including:

GDPR (General Data Protection Regulation) – Ensures user data privacy and consent.

PCI-DSS (Payment Card Industry Data Security Standard) – Secures credit card transactions.

CCPA (California Consumer Privacy Act) – Protects consumer rights regarding data usage.

KYC (Know Your Customer) & AML (Anti-Money Laundering) Regulations – Prevents fraudulent transactions and identity theft.

- **Feasible:** Compliance is achievable with strong data encryption, anonymization, and role-based access controls.

2. Ethical Considerations

User Privacy: The system must process sensitive financial data without compromising personal information.

Bias & Fairness: Machine learning models should be trained on diverse datasets to prevent discrimination against certain user groups.

Transparency: Users should be informed about fraud detection decisions and have a way to appeal false positives.

- **Feasible:** Ethical concerns can be mitigated through transparent AI practices and user data protection measures.

3. Data Security & Responsible AI Use

Secure storage (AES-256 encryption, tokenization) ensures unauthorized parties cannot access sensitive data.

Explainable AI (XAI) helps regulators and customers understand how fraud decisions are made.

Regular audits & compliance checks ensure continued legal and ethical alignment.

- **Feasible:** Implementing responsible AI principles and strict data security measures ensures ethical and legal compliance.

CHAPTER - 5

SYSTEM DESIGN

5.1 MTCNN

Architectural specifications are understood and designed in this phase. Usually more than one technical approach is proposed and based on the technical and financial feasibility the final decision is taken. System design is broken down further into modules taking up different functionality. This is also referred to as High Level Design HLD.

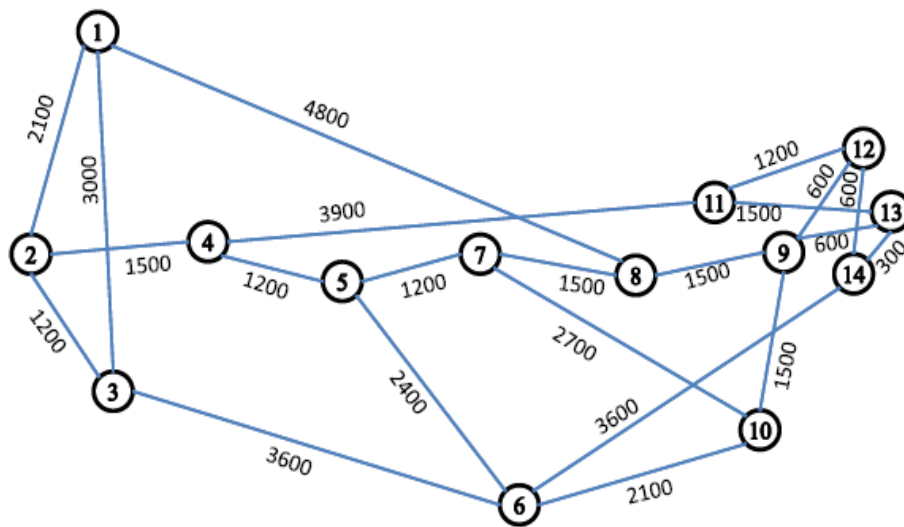
The data transfer and communication between the internal modules and with the outside world other systems is clearly understood and defined in this stage. With this information, integration tests can be designed and documented during this stage.

The most creative and challenging phase of the life cycle is system and design. The term design describes a final system and the process by which it is developed. It refers to the technical specifications that will be applied in implementation the candidate system. The design may be defined as –the process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient details to permit its physical realization.

System Structuring

The design's goal is how the output is to be produced and in what format samples of the output and input are also presented. Second input data and database files have to be designed to meet the requirements of the proposed output. The processing phase is handled through the program construction and testing. Finally, details related to justification of the system and an estimate of the impact of the candidate system on the users and the organization are documented and evaluated by management as a step toward implementation.

The importance of software design can be stated in a single word “**Quality**”. Design provides us with representation of software that can be assessed for quality. Design is the only way that we can accurately translate a customer's requirements into a finished software product or system without design we risk building an unstable system, that might fail if small changes are made or may be difficult to test, or one whose quality can't be tested. So, it is an essential phase in the development of a software product.



NSFNET topology with the lengths in km marked on links.

Fig 5.1 System Architecture

5.2 UML DIAGRAMS

Introduction to UML

UML is a method for describing the system architecture in detail using the blue print. UML implies a gathering of best building watches that have affirmed effective in the displaying of enormous and troublesome frameworks. The UML is an exceptionally noteworthy piece of creating articles situated programming and the product improvement process. The UML utilizes for the most part graphical documentations to positive the outline of programming tasks. Spending the UML tasks groups interconnect, find potential plans, and approve the structural configuration of the product.

Definition: UML is a general-purpose visual modeling language that is used to specify, visualize, construct, and document the artifacts of the software system.

UML is a language: It will provide vocabulary and rules for communications and function on conceptual and physical representation. So, it is modeling language.

UML Specifying: Specifying means building models that are precise, unambiguous and complete. In particular, the UML address the specification of all the important analysis, design and implementation decisions that must be made in developing and displaying a software intensive system.

UML Visualization: The UML includes both graphical and textual representation. It makes easy to visualize the system and for better understanding.

UML Constructing: UML models can be directly connected to a variety of programming languages and it is sufficiently expressive and free from any ambiguity to permit the direct execution of models.

UML Documenting: UML provides variety of documents in addition raw executable codes.

Uses of UML:

The UML is intended primarily for software intensive systems. It has been used effectively for such domain as

1. Enterprise Information System
2. Banking and Financial Services
3. Telecommunications
4. Transportation
5. Defense/Aerospace
6. Retails
7. Medical Electronics
8. Scientific Fields
9. Distributed Web

Rules of UML:

The UML has semantic rules for

SCOPE: The content that gives specific meaning to a name.

NAMES: It will call things, relationships and diagrams

VISIBILITY: How those names can be seen and used by others.

INTEGRITY: How things properly and consistently relate to another.

EXECUTION: What it means is to run or simulate a dynamic model.

Building blocks of UML:

The vocabulary of the UML encompasses 3 kinds of building blocks

- Things
- Relationships
- Diagrams

Things:

Things are the data abstractions that are first class citizens in a model. Things are of 4 types

- Structural Things
- Behavioral Things

- Grouping Things
- A notational Things

Relationships:

Relationships tie the things together. Relationships in the UML are

- Dependency
- Association
- Generalization
- Specialization

UML Diagrams:

Diagrams in the UML are of 2 types Static or Structural Diagrams

Static:

- Dynamic or Behavioral Diagrams
- Class diagrams
- Object diagrams.
- Component diagrams.

Dynamic:

- Deployment diagrams
- Use Case diagram.
- Sequence diagram
- Collaboration diagram.
- State Chart diagram.
- Activity diagram.

5.3 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

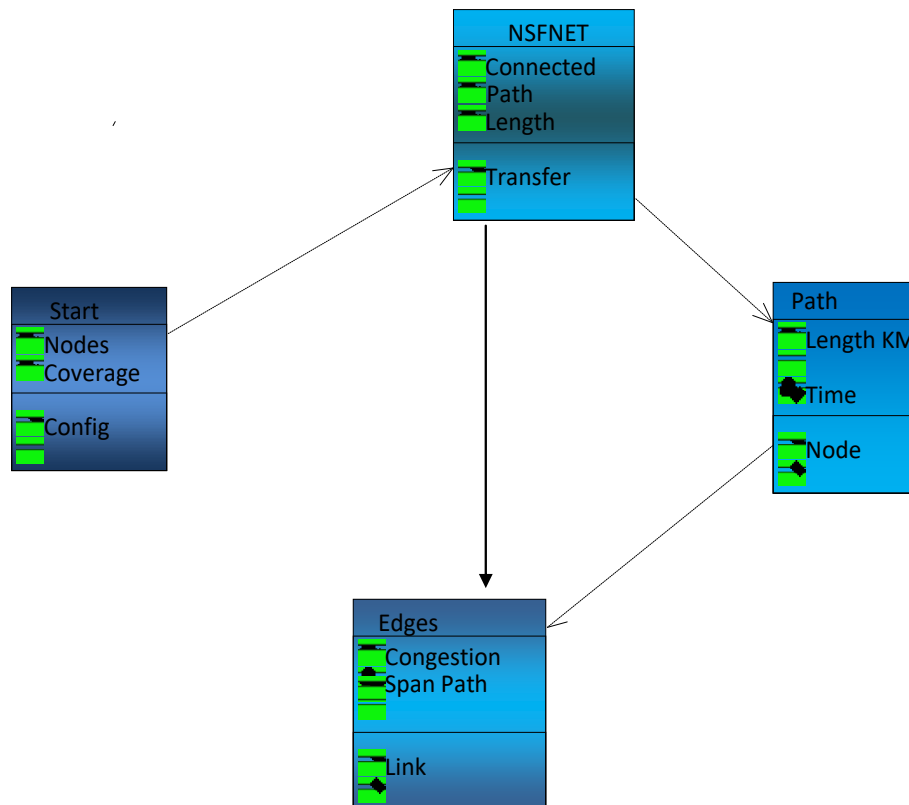
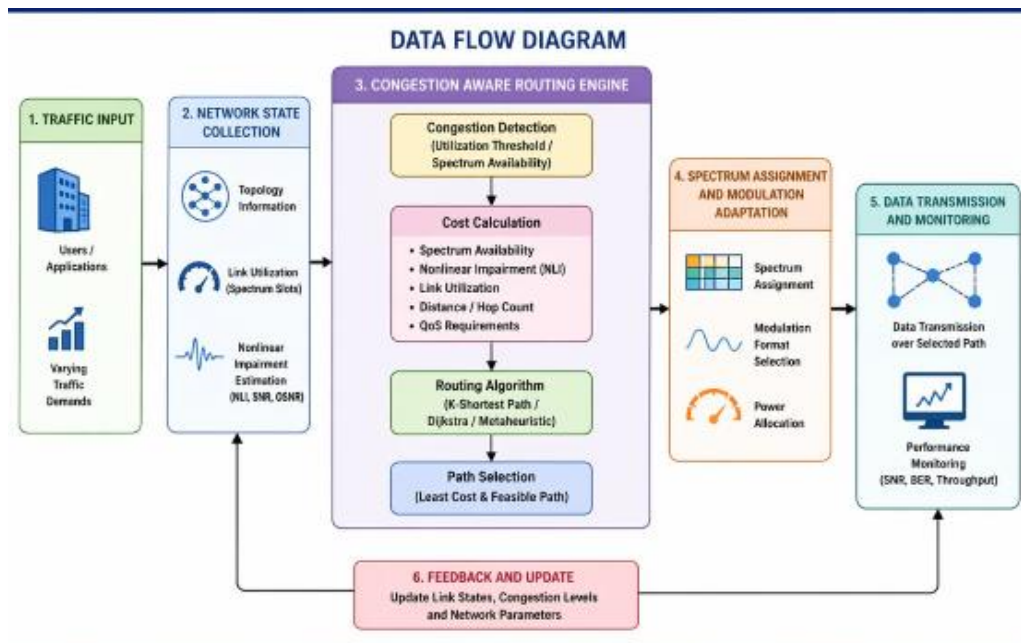


Fig 5.3 CLASS DIAGRAM

5.4 DATA FLOW DIAGRAM

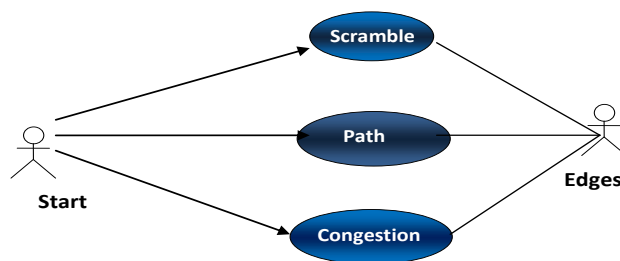
1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.



5.4 DATA FLOW DIAGRAM

5.5 USE CASE DIAGRAM:

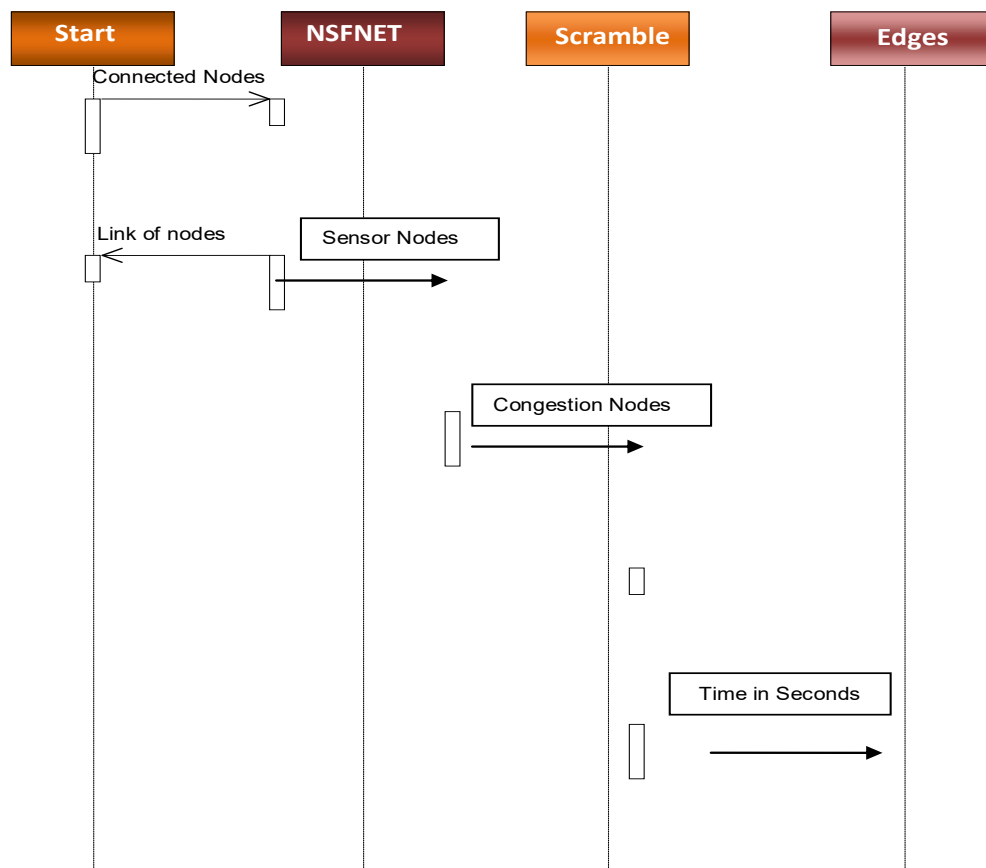
A use case diagram in the Unified Modeling Language (UML) is a kind of behavioral outline produced by and produced using a Use-contextual investigation. Its determination is to surviving a graphical sign of the usefulness giving by a framework regarding performing artists and their points (spoke to as use cases), and any conditions between those utilization cases. The key reason for a utilization case outline is to show what framework capacities are performed for which on-screen character. Parts of the on-screen characters in the framework can be delineated.



5.5 USER CASE DIAGRAM

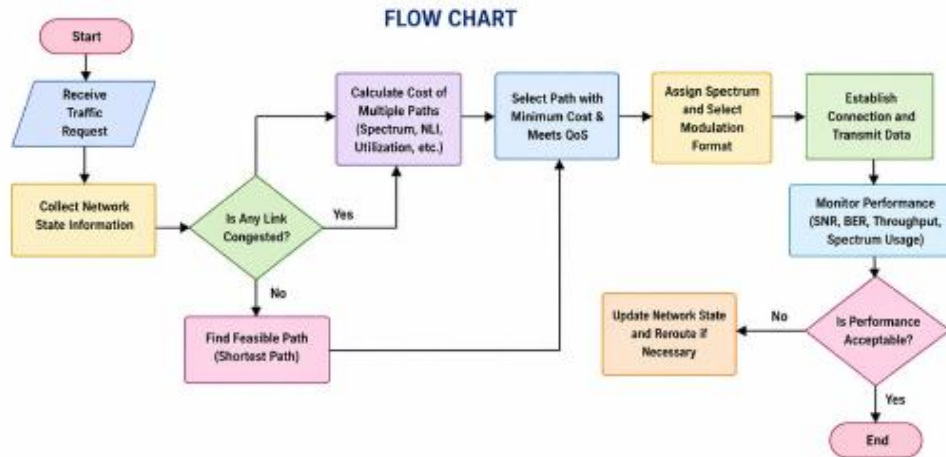
5.6 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



5.6 SEQUENCE DIAGRAM

5.7 FLOWCHAT



5. 7 Flow chat

SOFTWARE REQUIREMENTS:

The software requirements specification is produced at the end of the analysis task. Software requirement is a difficult task, only decided after testing whether it fits the requirements.

- Operating System - Windows XP
- Coding Language - Java/J2EE
- Front End - J2EE
- Back End - MySQL

HARDWARE REQUIREMENTS:

This is a java project so hardware plays an important role. Selection of hardware also plays an important role in existence and performance of any software. The size and capacity are main requirements.

- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

CHAPTER - 6

SYSTEM IMPLEMENTATION

6.1 TECHNOLOGY DESCRIPTION:

About Front-End

Java Technology

Java technology is both a programming language and a platform.

The Java Programming Language

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called Java byte codes —the platform-independent codes interpreted by the interpreter on the Java platform. The interpreter parses and runs each Java byte code instruction on the computer. Compilation happens just once; interpretation

occurs each time the program is executed. The following figure illustrates how this works.

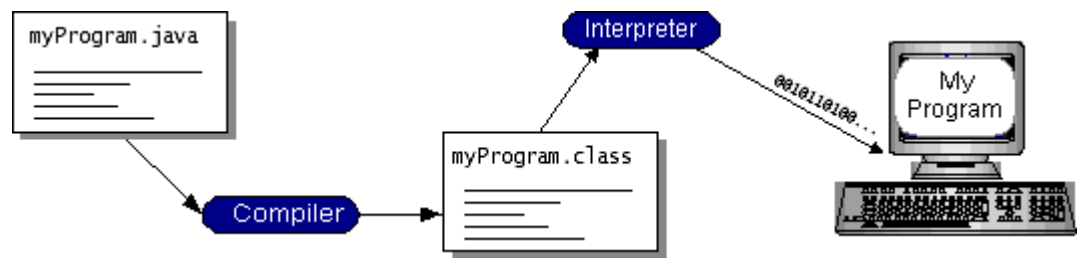


Fig 6.1 java programing

You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java byte codes help make -write once, run anywhere! possible. You can compile your program into byte codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM. That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.

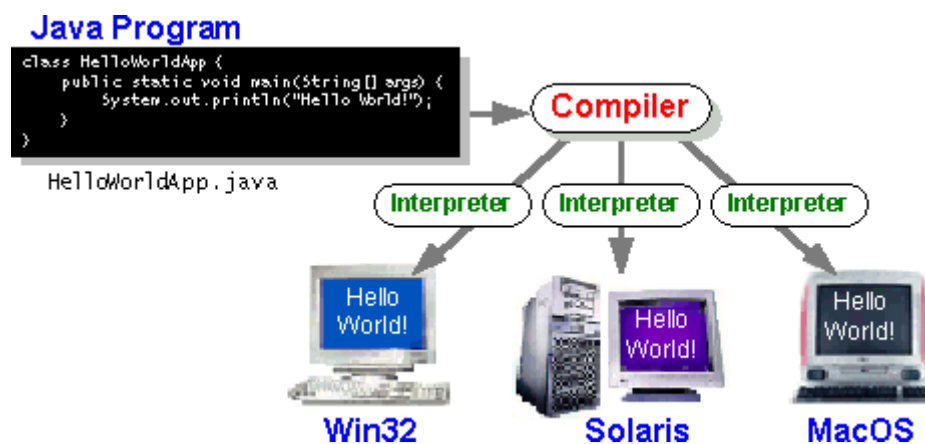


Fig 6.2 java program

The Java Platform

A platform is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating

system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware-based platforms.

The Java platform has two components:

- The Java Virtual Machine (Java VM)
- The Java Application Programming Interface (Java API)

You've already been introduced to the Java VM. It's the base for the Java platform and is ported onto various hardware-based platforms.

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and interfaces; these libraries are known as packages. The next section, What Can Java Technology Do? Highlights what functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.

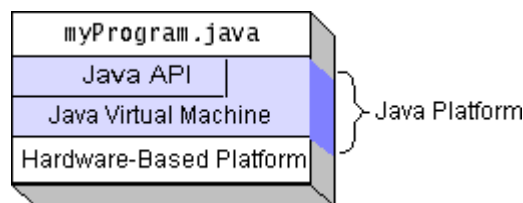


Fig 6.3 java plat form

Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart compilers, well-tuned interpreters, and just-in-time byte code compilers can bring performance close to that of native code without threatening portability.

What Can Java Technology Do?

The most common types of programs written in the Java programming language are applets and applications. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser.

However, the Java programming language is not just for writing cute, entertaining applets

for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

An application is a standalone program that runs directly on the Java platform. A special kind of application known as a server serves and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a servlet. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- **The essentials:** Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- **Applets:** The set of conventions used by applets.
- **Networking:** URLs, TCP (Transmission Control Protocol), UDP (User Datagram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization:** Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.
- **Security:** Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components:** Known as JavaBeansTM, can plug into existing component architectures.
- **Object serialization:** Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- **Java Database Connectivity (JDBCTM):** Provides uniform access to a wide range of relational databases.

The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more. The following figure depicts what is included in the Java 2 SDK.

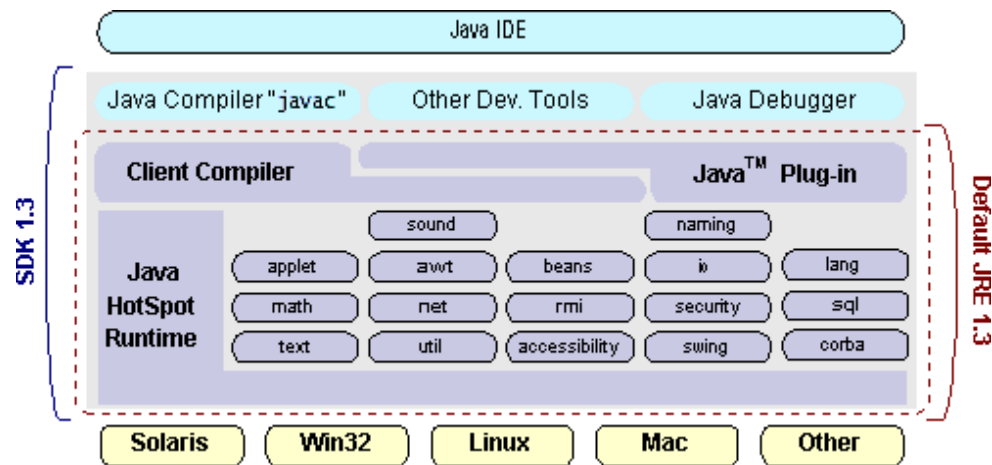


Fig 6.4 java ide

How Will Java Technology Change My Life?

We can't promise you fame, fortune, or even a job if you learn the Java programming language. Still, it is likely to make your programs better and requires less effort than other languages. We believe that Java technology will help you do the following:

- **Get started quickly:** Although the Java programming language is a powerful object-oriented language, it's easy to learn, especially for programmers already familiar with C or C++.
- **Write less code:** Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.
- **Write better code:** The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory leaks. Its object orientation, its JavaBeans component architecture, and its wide-ranging, easily extendible API let you reuse other people's tested code and introduce fewer bugs.
- **Develop programs more quickly:** Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.

- **Avoid platform dependencies with 100% Pure Java:** You can keep your program portable by avoiding the use of libraries written in other languages. The 100% Pure Java™ Product Certification Program has a repository of historical process manuals, white papers, brochures, and similar materials online.
- **Write once, run anywhere:** Because 100% Pure Java programs are compiled into machine-independent byte codes, they run consistently on any Java platform.
- **Distribute software more easily:** You can upgrade applets easily from a central server. Applets take advantage of the feature of allowing new classes to be loaded —on the fly, without recompiling the entire program.

About Front End

ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a de facto standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC data sources. From a programming perspective, the beauty of ODBC is that the

application can be written to use the same set of function calls to interface with any data source, regardless of the database vendor. The source code of the application doesn't change whether it talks to Oracle or SQL Server. We only mention these two as an example. There are ODBC drivers available for several dozen popular database systems. Even Excel spreadsheets and plain text files can be turned into data sources. The operating system uses the Registry information written by ODBC Administrator to determine which low-level ODBC drivers are needed to talk to the data source (such as the interface to Oracle or SQL Server). The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server environment, the ODBC API even handles many of the network issues for the application programmer.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface. ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year.

JDBC

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of -plug-in database connectivity modules, or drivers. If a database vendor wishes to have JDBC support, he or she must provide the driver for each platform that the database and Java run on.

To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution.

JDBC was announced in March of 1996. It was released for a 90 day public review that

ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

JDBC Goals

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java.

The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do. The eight design goals for JDBC are as follows:

1. SQL Level API

The designers felt that their main goal was to define a SQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently. Attaining this goal allows for future tool vendors to —generate JDBC code and to hide many of JDBC's complexities from the end user.

2. SQL Conformance

SQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed through it to the underlying database driver. This allows the connectivity module to handle non-standard functionality in a manner that is suitable for its users.

3. JDBC must be implemental on top of common database interfaces The JDBC SQL API must sit on top of other common SQL level APIs. This goal allows JDBC to use existing ODBC level drivers by the use of a software interface. This interface would translate JDBC calls to ODBC and vice versa.

4. Provide a Java interface that is consistent with the rest of the Java system

Because of Java's acceptance in the user community thus far, the designers feel that they should not stray from the current design of the core Java system.

5. Keep it simple

This goal probably appears in all software design goal listings. JDBC is no exception. Sun felt that the design of JDBC should be very simple, allowing for only one method of completing a task per mechanism. Allowing duplicate functionality only serves to confuse the users of the API.

6. Use strong, static typing wherever possible

Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

7. Keep the common cases simple

Because more often than not, the usual SQL calls used by the programmer are simple SELECT's, INSERT's, DELETE's and UPDATE's, these queries should be simple to perform with JDBC. However, more complex SQL statements should also be possible.

Finally, we decided to proceed the implementation using Java Networking.

And for dynamically updating the cache table we go for MS Access database.

Java ha two things: a programming language and a platform. Java is a high-level programming language that is all of the following

Simple	Architecture-neutral Object-oriented
Portable	
Distributed	High-performance
Interpreted	multithreaded Robust
Dynamic	
Secure	

Java is also unusual in that each Java program is both compiled and interpreted. With a compiler you translate a Java program into an intermediate language called Java byte codes. The platform-independent code instruction is passed and run on the computer.

Compilation happens just once; interpretation occurs each time the program is executed. The figure illustrates how this works.

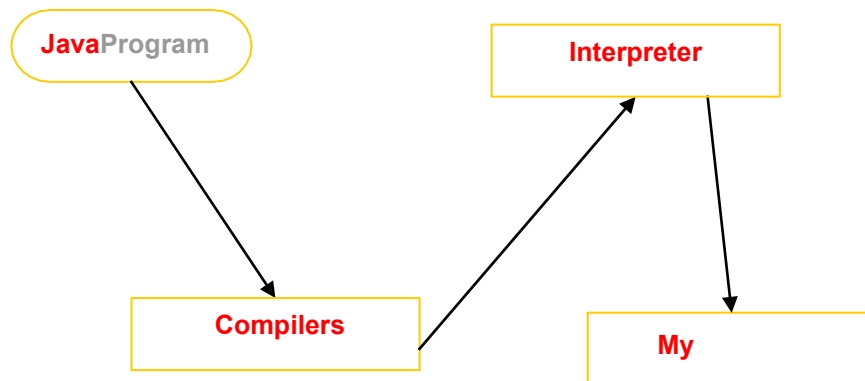


Fig 6.5 java compiler

You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a Java development tool or a Web browser that can run Java applets, is an implementation of the Java VM. The Java VM can also be implemented in hardware.

Java byte codes help make –write once, run anywhere possible. You can compile your Java program into byte codes on my platform that has a Java compiler. The byte codes can then be run any implementation of the Java VM. For example, the same Java program can run Windows NT, Solaris, and Macintosh.

NetworkingTCP/IP stack

The TCP/IP stack is shorter than the OSI one:

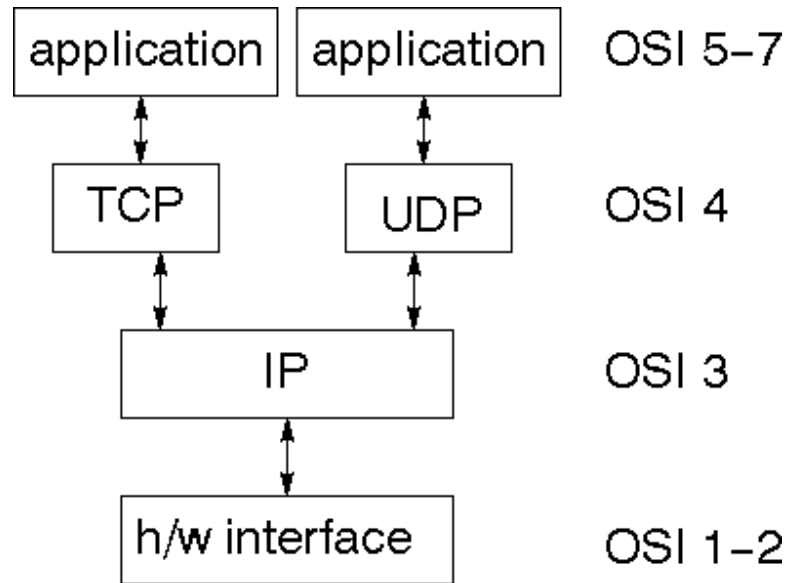


Fig 6.6 networking TCP/IP

TCP is a connection-oriented protocol; UDP (User Datagram Protocol) is a connectionless protocol.

IP datagram's

The IP layer provides a connectionless and unreliable delivery system. It considers each datagram independently of the others. Any association between datagram must be supplied by the higher layers. The IP layer supplies a checksum that includes its own header. The header includes the source and destination addresses. The IP layer handles routing through an Internet. It is also responsible for breaking up large datagram into smaller ones for transmission and reassembling them at the other end.

UDP

UDP is also connectionless and unreliable. What it adds to IP is a checksum for the contents of the datagram and port numbers. These are used to give a client/server model - see later.

TCP

TCP supplies logic to give a reliable connection-oriented protocol above IP. It provides a virtual circuit that two processes can use to communicate.

Internet addresses

In order to use a service, you must be able to find it. The Internet uses an address scheme for machines so that they can be located. The address is a 32 bit integer which gives the IP address. This encodes a network ID and more addressing. The network ID falls into various classes according to the size of the network address.

Network address

Class A uses 8 bits for the network address with 24 bits left over for other addressing. Class B uses 16 bit network addressing. Class C uses 24-bit network addressing and class D uses all 32.

Subnet address

Internally, the UNIX network is divided into sub networks. Building 11 is currently on one sub network and uses 10-bit addressing, allowing 1024 different hosts.

Host address

8 bits are finally used for host addresses within our subnet. This places a limit of 256 machines that can be on the subnet.

Total address

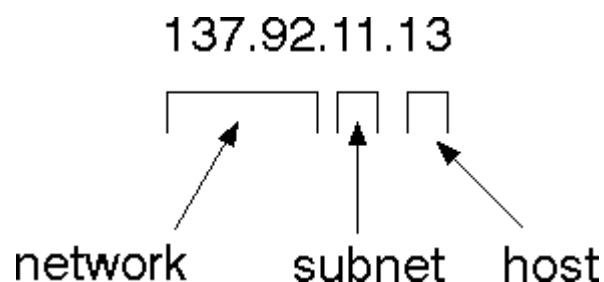


Fig 6.7 total address

The 32-bit address is usually written as 4 integers separated by dots.

Port addresses

A service exists on a host, and is identified by its port. This is a 16 bit number. To send a message to a server, you send it to the port for that service of the host that it is running on. This is not location transparency! Certain of these ports are "well known".

Sockets

A socket is a data structure maintained by the system to handle network connections. A socket is created using the call `socket`. It returns an integer that is like a file descriptor. In fact, under Windows, this handle can be used with `Read File` and `Write File` functions.

```
#include <sys/types.h> #include <sys/socket.h>
int socket(int family, int type, int protocol);
```

Here "family" will be `AF_INET` for IP communications, protocol will be zero, and type will depend on whether TCP or UDP is used. Two processes wishing to communicate over a network create a socket each. These are similar to two ends of a pipe - but the actual pipe does not yet exist.

JFree Chart

JFreeChart is a free 100% Java chart library that makes it easy for developers to display professional quality charts in their applications. JFreeChart's extensive feature set includes:

A consistent and well-documented API, supporting a wide range of chart types;

A flexible design that is easy to extend, and targets both server-side and client-side applications;

Support for many output types, including Swing components, image files (including PNG and JPEG), and vector graphics file formats (including PDF, EPS and SVG);

JFreeChart is "open source" or, more specifically, free software. It is distributed under the terms of the GNU Lesser General Public [Licence](#) (LGPL), which permits use in proprietary applications.

1. Map Visualizations

Charts showing values that relate to geographical areas. Some examples include: (a) population density in each state of the United States, (b) income per capita for each country in Europe, (c) life expectancy in each country of the world. The tasks in this project include:

Sourcing freely redistributable vector outlines for the countries of the world, states/provinces in particular countries (USA in particular, but also other areas);

Creating an appropriate dataset interface (plus default implementation), a rendered, and integrating this with the existing XYPlot class in JFreeChart;

Testing, documenting, testing some more, documenting some more.

2. Time Series Chart Interactivity

Implement a new (to JFreeChart) feature for interactive time series charts --- to display a separate control that shows a small version of ALL the time series data, with a sliding "view" rectangle that allows you to select the subset of the time series data to display in the main chart.

3. Dashboards

There is currently a lot of interest in dashboard displays. Create a flexible dashboard mechanism that supports a subset of JFreeChart chart types (dials, pies, thermometers, bars, and lines/time series) that can be delivered easily via both Java Web Start and an applet.

4. Property Editors

The property editor mechanism in JFreeChart only handles a small subset of the properties that can be set for charts. Extend (or reimplement) this mechanism to provide greater end-user control over the appearance of the charts.

Tomcat 6.0 web server

Tomcat is an open-source web server developed by Apache Group. Apache Tomcat is the

servlet container that is used in the official Reference Implementation for the Java Servlet and JavaServer Pages technologies. The Java Servlet and JavaServer Pages specifications are developed by Sun under the Java Community Process. Web Servers like Apache Tomcat support only web components while an application server supports web components as well as business components (BEAs Weblogic, is one of the popular application servers). To develop a web application with jsp/servlet install any web server like JRun, Tomcat etc to run your application.

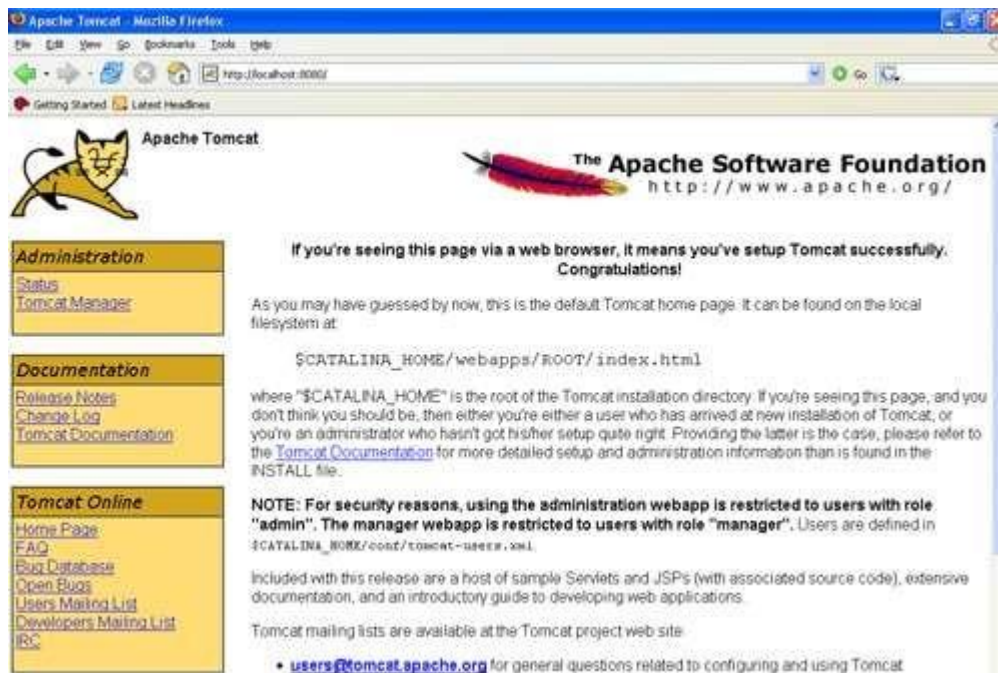


Fig 6.8 Tomcat Webserver

6.2 MODULAR DESCRIPTION

Implementation is the stage of the project when the theoretical design is turned out into a working system. Thus, it can be considered to be the most critical stage in achieving a successful new system and in giving the user, confidence that the new system will work and be effective.

Implementation stage involves careful planning, investigation of the existing system and its constraints on implementation, designing of methods to achieve changeover and evaluation of changeover methods.

FIBER NONLINEARITIES:

Fiber nonlinear optical network the RMSA becomes primarily a routing problem, with the assignment of modulation format and forward error correction (FEC) depending on the signal to noise available for a given route and the optical spectrum to be assigned depending on the required data rate. Given the increased importance of the routing algorithm for the nonlinear RMSA, in this letter we investigate the impact of congestion aware routing algorithms and quantify their performance and efficacy in delaying the onset of network blocking for the NSFNET topology.

NETWORK PROBABILITY (NBP):

Network blocking probability (NBP), we follow the methodology of sequentially loading the network with bi-directional demands between randomly selected pairs of nodes in the network. In this letter for simplicity, we restrict our discussion to uniformly generated traffic with a demand granularity of 100 GbE, each of which is independently optically routed through the network. The point at which the path assigned by the routing algorithm cannot physically be optically routed by the network is considered blocking and the number of demands recorded. This procedure is then repeated 10000 times to build up the statistical behavior of the network blocking probability.

ROUTING ALGORITHMS (CAR):

Congestion aware routing we make the following assumptions in our analysis:

1. The client-side data rate is fixed (herein we restrict our consideration to 100 GbE assumed to be 104 Gbit/s including framing plus an overhead for FEC).
2. Transceivers can vary modulation format and FEC.
3. Channels are Nyquist shaped, having a rectangular spectrum of width equal to the symbol rate.
4. There are negligible guard bands between channels.
5. The network employs single mode fiber and is periodically amplified by a lumped erbium doped fiber amplifier (EDFA) with a bandwidth of 5 THz.

FIBRE NONLINEAR RMSA:

The proposed algorithm for RMSA in a nonlinear elastic network utilizing Nyquist pulse shaping is as follows:

1. Determine the optimum signal power spectral density given the fiber and amplifier parameters.

2. For a pair of nodes, select the shortest path that avoids the link with the highest spectral usage (determined by measuring the total optical power which is proportional to spectral usage).
3. For this path determine the total number of amplifier spans (100 km herein) in order to determine the received signal to noise ratio (SNR).
4. For this SNR, determine the maximum net spectral efficiency (NSE) based on known relationship between SNR and NSE for a range of polarization division multiplexed formats with Nyquist spectra where variable rate FEC is also included.
5. Finally determine the gross symbol rate and assign spectrum to serve the demand between the two nodes.

CHAPTER – 7

TESTING & DEBUGGING

7.1 TESTING METHODOLOGIES

The goal of testing is to acquire errors. Testing is that the technique of trying to get each possible error or weakness in an extremely work product. It provides the way to observe the practicality of parts, sub-assemblies, assemblies and or a finished product it is the technique of effort code with the concentrating of guaranteeing that the software meets its requirements and user hopes and does not fail in an undesirable manner. There are numerous sorts of check. Every check sort reports a designated testing demand.

Testing objectives:

The key objective of testing is to determine a mass of errors, systematically and with minimum effort and time. Stating formally, we can say, testing is a process of executing a program with resolved of discover an error.

- A successful test is one that determines an as however undiscovered error.
- A good test case is one that has possibility of discover an error, if it exists.
- The test is insufficient to detect possibly present errors.
- The software more or less approves to the quality and unswerving standards.

Types of Testing

The basic levels of Testing:

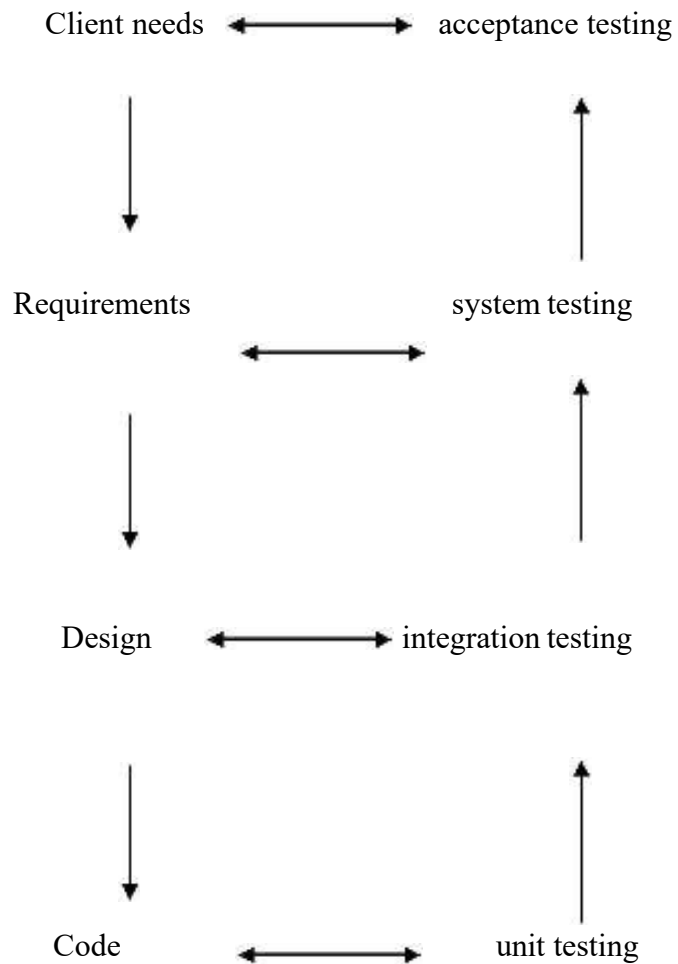


Fig. 7.1 Levels of Testing

Unit Testing & Test Cases

1. Unit testing:

A unit is the smallest piece of source code that can be tested. It is also known as a module which consists of numerous lines of code that are processed by a single programmer. The key purpose of performing unit testing is to expose that a particular unit doesn't satisfy the specified functional requirements and also to show that the structural implementation is not like to the projected structure designed.

1. Integration Testing & Test Cases

Integration testing:

Integration tests are intended to test incorporated programming segments to figure out whether they really keep running as one system. Testing is occasion driven and is more worried with the fundamental result of screens or fields. Reconciliation tests exhibit that in spite of the fact that the parts were separately fulfillment, as appeared by effectively unit testing, the blend of segments is right and comprised. Integration testing is specifically aimed at revealing the problems that rise from the mixture of components.

2. User Acceptance Testing & Test Cases

User Acceptance Testing is a serious phase of any project and needs important contribution by the end user. It also guarantees that the system encounters the functional requirements.

3. Output Testing & Test Cases

Test strategy and approach

Ground testing will be done physically and functional tests will be inscribed in detail.

Test objectives

- All field admissions essentially work appropriately.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Validate that the accesses are of the correct format

- No duplicate entries should be allowed
- Entire links must gross the user to the accurate page.

4. Validation Testing & Test Cases

Real tests give efficient protests that functions tested are attainable as specific by the business and technical requirements, system documentation, and user manuals.

Functional testing is focused on the subsequent items:

Valid Input : known categories of valid input should be accepted. Invalid Input: known categories of invalid input should be rejected.

Functional test

Functional tests give efficient challenges that capacities tried are accessible as determined by the business and specialized necessities, framework documentation, and client manuals Functional testing is centered on the following items:

Valid Input: identified classes of valid input must be accepted.

Invalid Input: identified classes of invalid input must be rejected.

Functions: identified functions must be exercised.

Output: identified classes of application outputs must be exercised. Systems/Procedures: interfacing systems or procedures must be invoked.

Association and arrangement of practical tests is centered around prerequisites, key capacities, or unique experiments. Likewise, efficient scope relating to recognize Business procedure streams; information fields, predefined procedures, and progressive procedures must be considered for testing. Before utilitarian testing is finished, extra tests are distinguished and the powerful estimation of current tests is resolved.

SYSTEM SECURITY

System testing guarantees that the entire coordinated programming framework meets prerequisites. It tests a design to guarantee known and unsurprising results. A sample of framework testing is the arrangement situated framework combination test. Framework testing depends on procedure portrayals and streams, stressing pre-driven procedure connections and mix focuses.

White Box Testing

It is a testing in which the product analyzer has information of the internal workings,

structure and dialect of the product, or if nothing else its motivation. It is reason. It is utilized to test ranges that can't be gotten a handle on from a discovery level.

Black Box Testing

It is the testing the product with no information of within workings, structure or dialect of the part being tried. Discovery tests, as most different sorts of tests, must be composed from a complete source report, for example, prerequisite or necessities archive, for example, determination or necessities record. It is a trying in which the product under test is dealt with, as a discovery. you can't "see" into it. The test gives inputs and reacts to yields without considering how the product functions.

Unit Testing:

Unit testing is by and large appeared as a major aspect of a joined code and unit test period of the product lifecycle, in spite of the fact that it is not exceptional for coding and unit testing to be directed as two unmistakable stages.

Integration Testing

Software integration testing is the incremental combination analysis of two or more joint software components on a single platform to generate failures created by boundary faults.

The task of the integration test is to design those components or s/w applications, e.g. modules in a software system or – one step up – software presentations at the company level – interact without faults.

Test Results: All the test cases stated above passed effectively. No defects met.

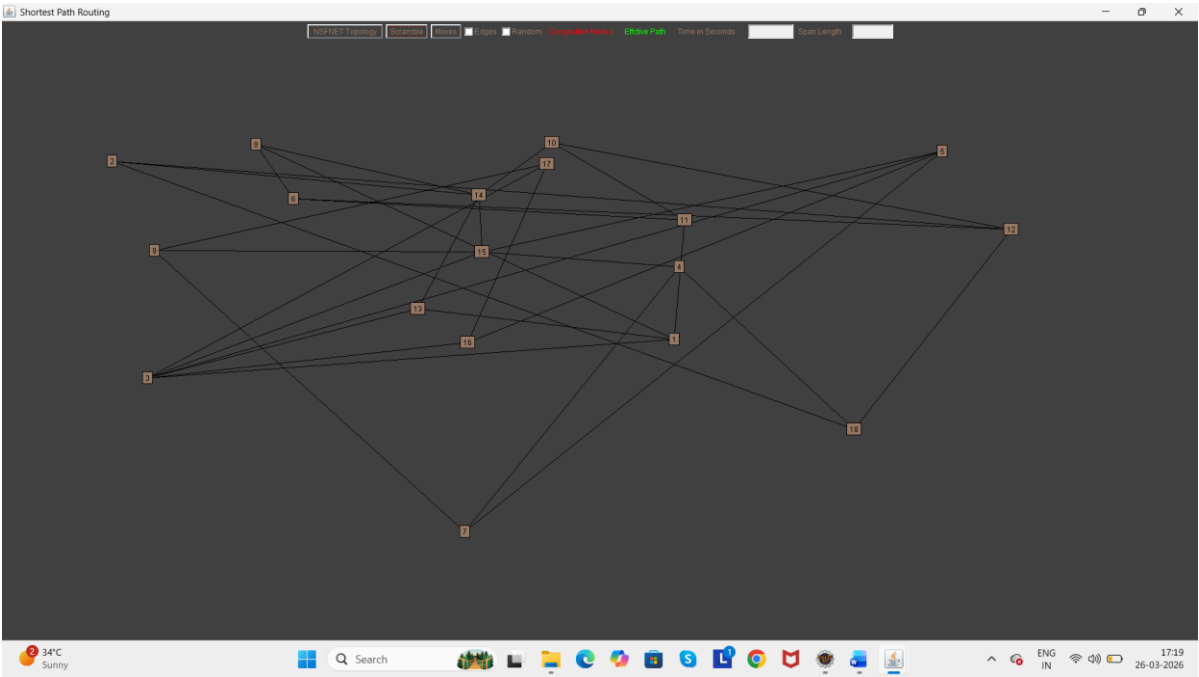
7.2 SYSTEM ENHANCEMENT

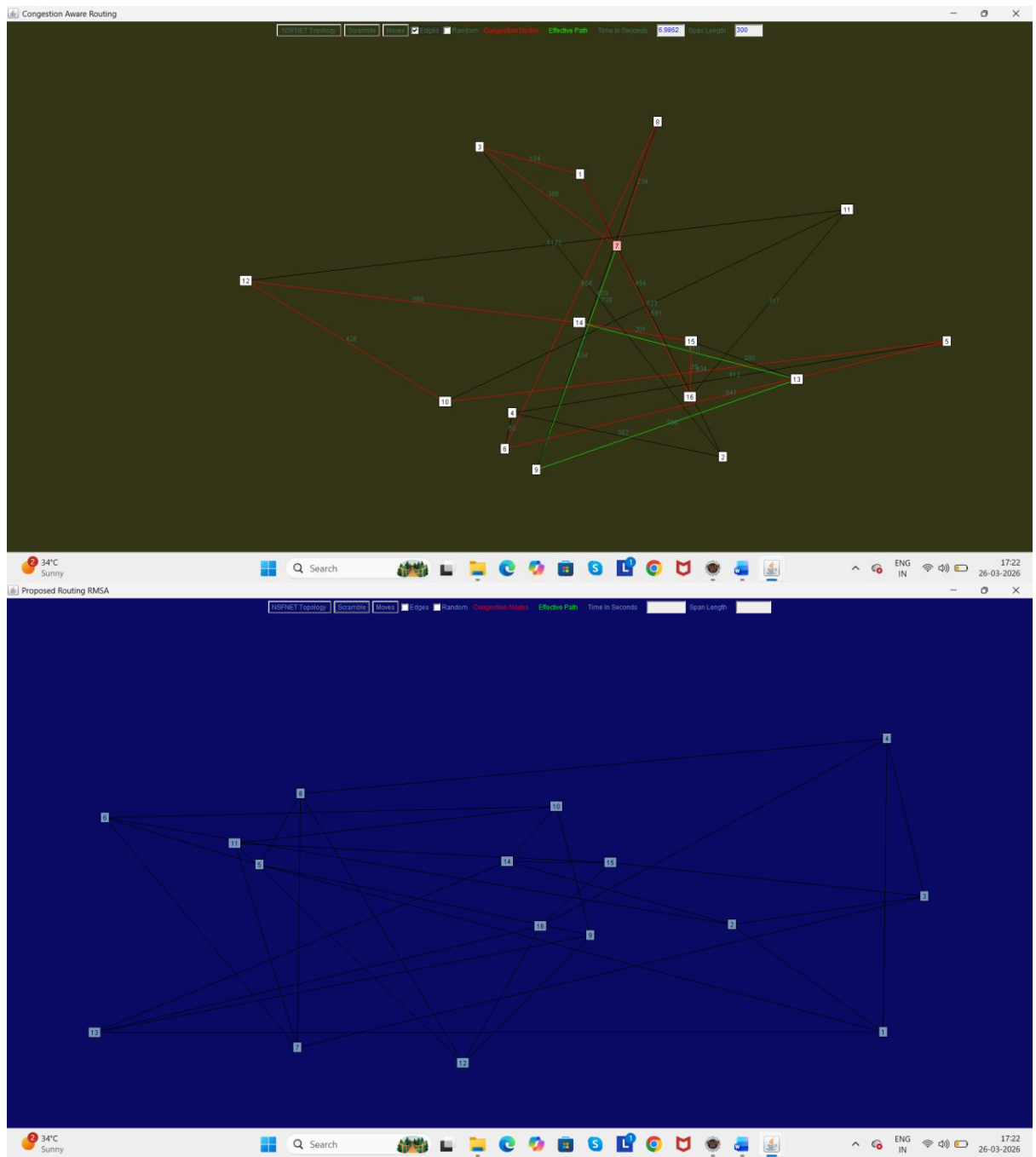
Test case id	Test case description	Actual value	Entered value	Status
1	Register user details in registration page	Fill all the fields while registering user	All the fields are filled	Pass
2	Give user name in text box	User name must be given in alphabets	User name given in alphabets and numeric values	Fail
3	Password to be entered in password box	Password must be given correctly	Password is entered wrongly	Fail
4	Phone number must be entered in phone number box during registration	Phone number must be given in 10 digits	Phone number given in 10 digits	Pass
5	Validating the functionality of Browse button	System should select the corresponding file	selected the file what we expected	Pass

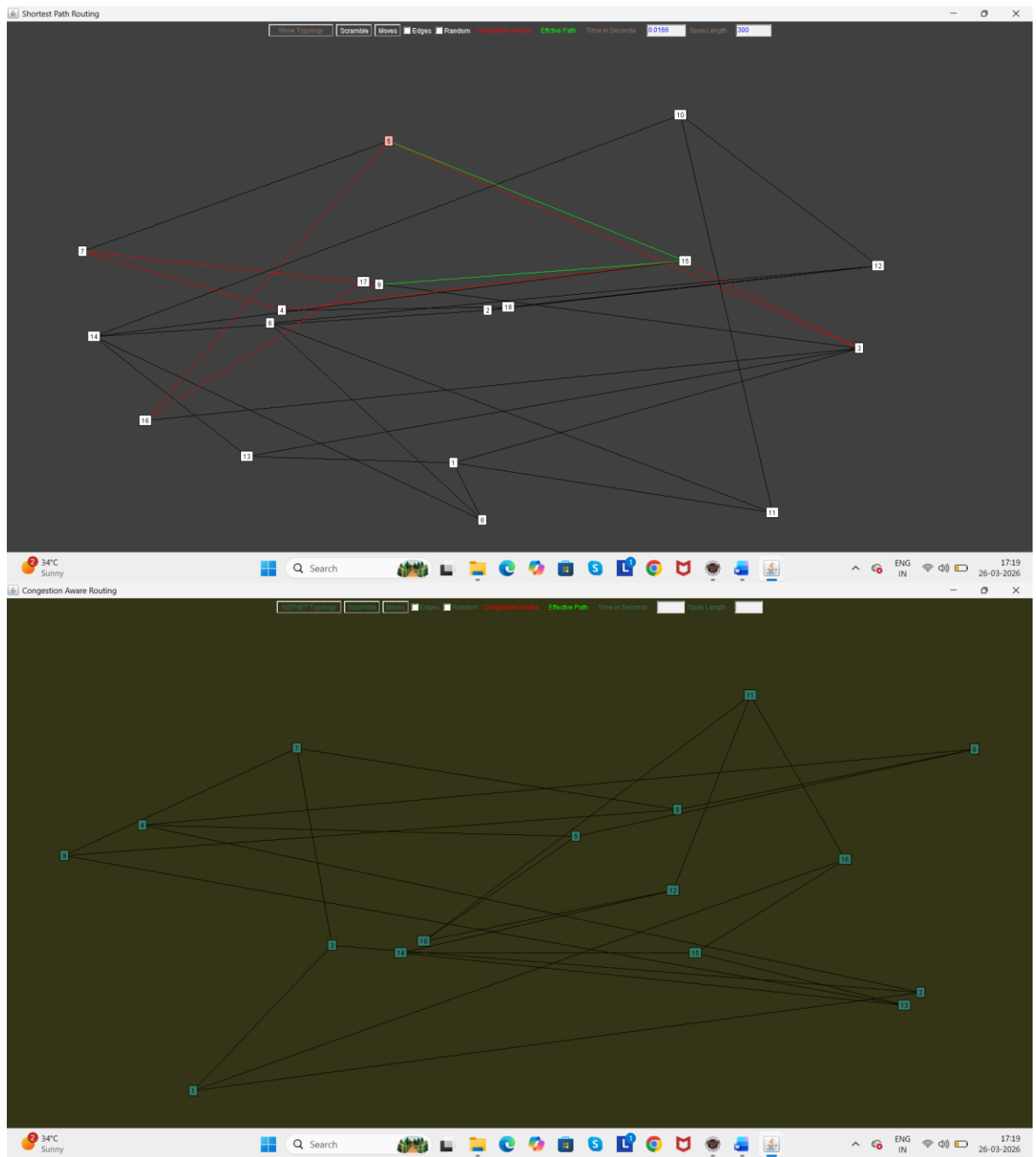
Fig 7.2: Testcase Template

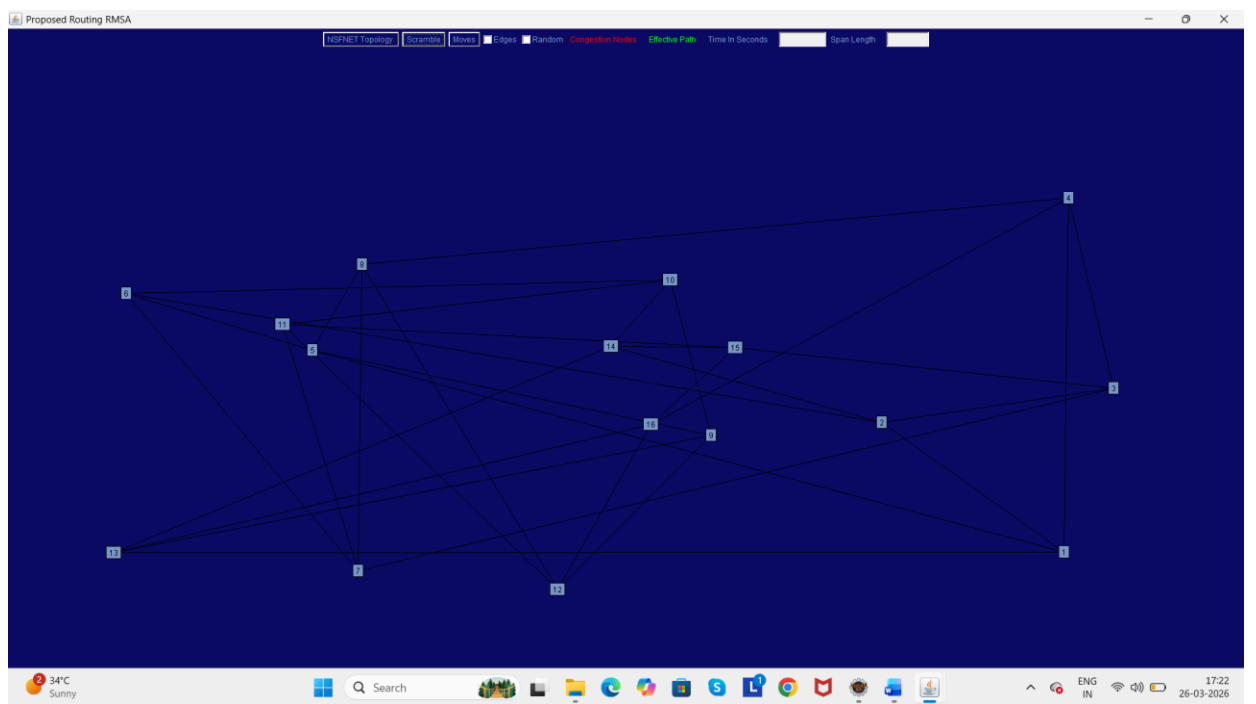
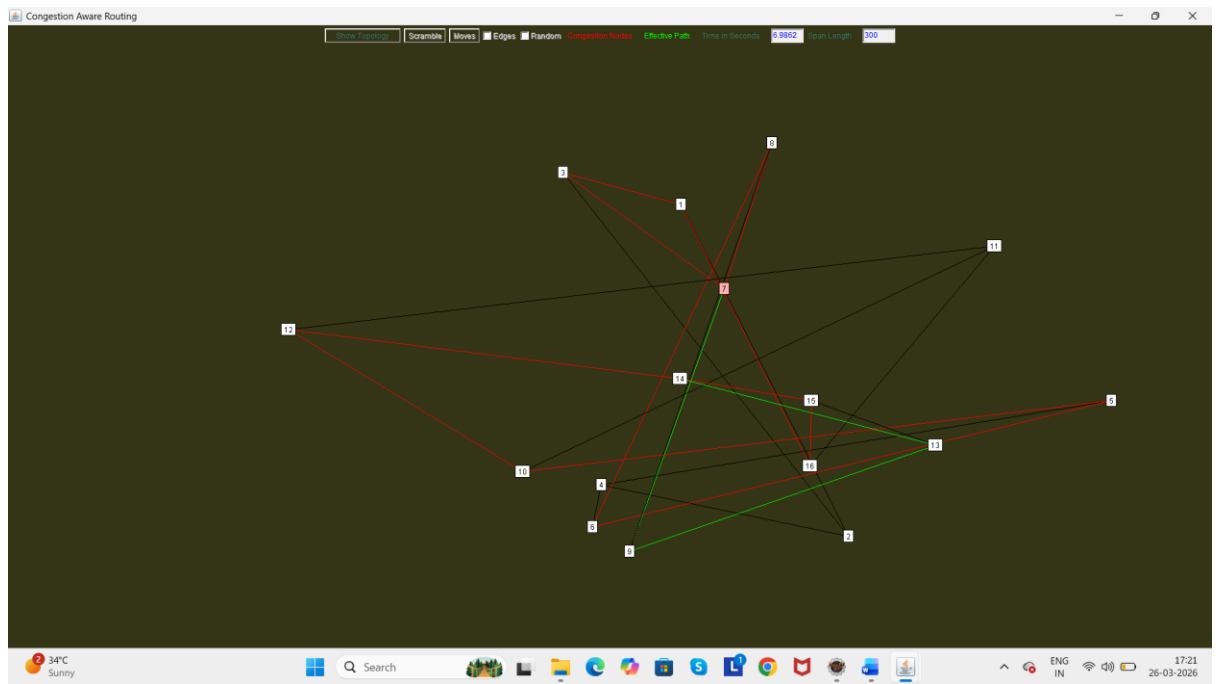
Test Results: All the test cases stated above passed effectively. No defects met.

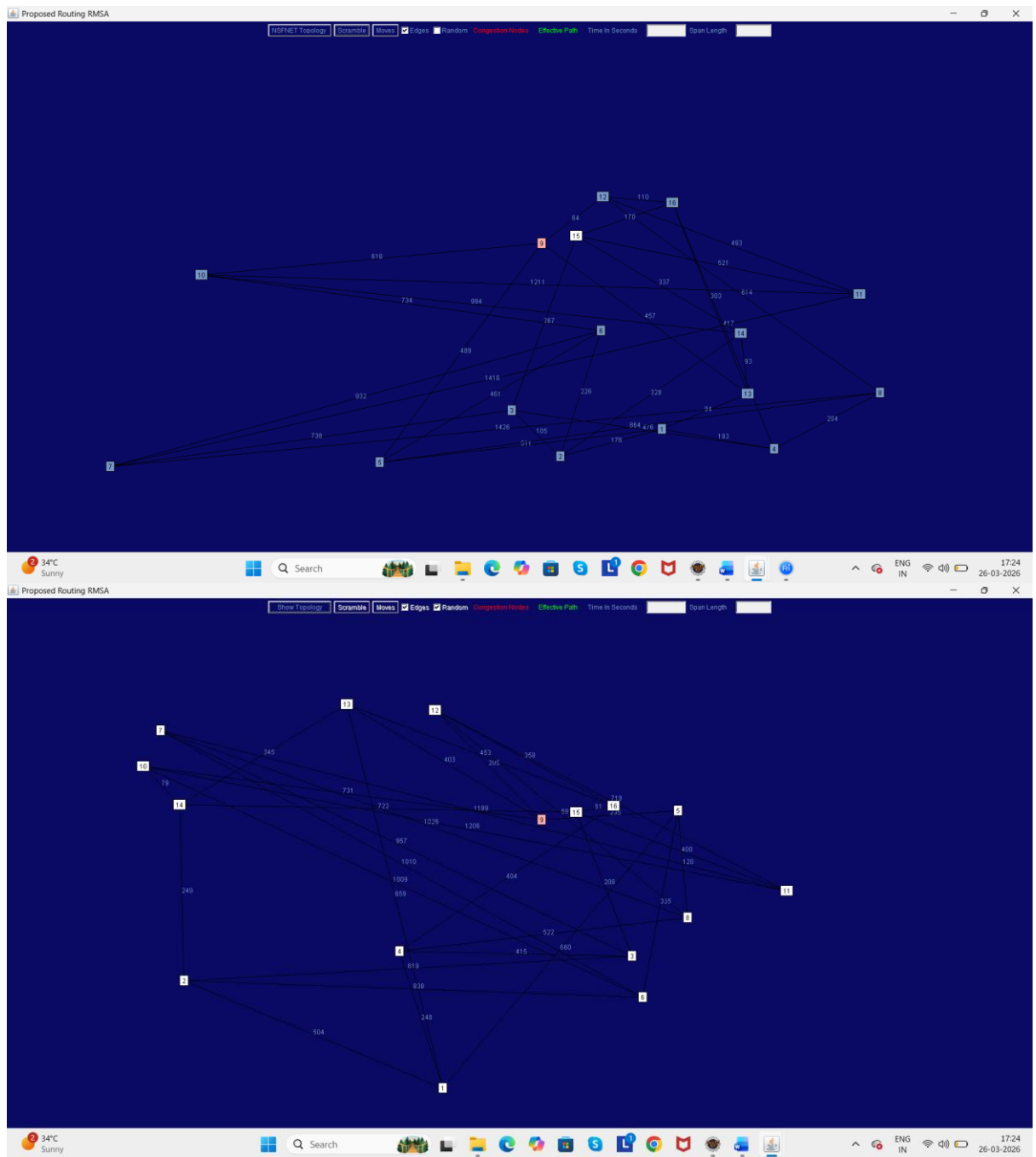
7.3 OUTPUT SCREENS:











CHAPTER - 8

CONCLUSION

Congestion aware routing has been investigated in nonlinear elastic optical networks and shown to be effective for the reference NSFNET topology. We observe that the network blocking probability (NBP) follows a generalized extreme value distribution, allowing robust estimates of the load for a given NBP to be obtained. When NSFNET is sequentially loaded with 100 GbE demands the proposed algorithm with a flexgrid, allows the network to support 1744 demands compared to 328 demands using a fixed 50 GHz grid with shortest path routing for $NBP = 1\%$. The congestion aware routing algorithms investigated resulted in longer average paths, with 5% of all routes exceeding the maximum shortest path in order to increase the overall network capacity.

CHAPTER - 9

FUTURE ENHANCEMENT

In future, the proposed congestion aware routing approach can be enhanced by incorporating real-time network monitoring to dynamically adjust routing decisions based on current traffic and congestion levels. The model can be extended to support larger and more complex network topologies beyond NSFNET for better scalability. Additionally, integrating machine learning techniques can help predict traffic patterns and optimize routing more efficiently. Further improvements can include advanced modulation formats and adaptive spectrum allocation strategies to enhance capacity and performance. Finally, efforts can be made to reduce the increase in path length while maintaining low blocking probability, ensuring a better balance between efficiency and network performance.

CHAPTER - 10

BIBLIOGRAPHY

- [1] O. Gerstel, M. Jinno, A. Lord, and S. J. B. Yoo, “Elastic optical networking: A new dawn for the optical layer?” *IEEE Commun. Mag.*, vol. 50, no. 2, pp. s12–s20, Feb. 2012.
- [2] L. Velasco, M. Klinkowski, M. Ruiz, and J. Comellas, “Modeling the routing and spectrum allocation problem for flexgrid optical networks,” *Photon. Netw. Commun.*, vol. 24, no. 3, pp. 177–186, 2012.
- [3] M. Jinno et al., “Distance-adaptive spectrum resource allocation in spectrum-sliced elastic optical path network,” *IEEE Commun. Mag.*, vol. 48, no. 8, pp. 138–145, Aug. 2010.
- [4] M. Angelou, P. N. Ji, I. Tomkos, and T. Wang, “QoT prediction for core networks with uncompensated coherent transmission,” in *Proc. OECC/PS Jul. 2013*, pp. 1–2, paper TuQ3-4.
- [5] A. Splett, C. Kurtzke, and K. Petermann, “Ultimate transmission capacity of amplified optical fiber communication systems taking into account fiber nonlinearities,” in *Proc. ECOC*, 1993, pp. 41–44.